

Get TradingAgents Running in Minutes

Follow this step-by-step guide to set up the multi-agent LLM financial trading framework on your machine.

Python 3.12+ pip install Docker optional ⌚ ~5 minutes

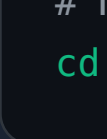
PROGRESS

0%

[Clone](#) [Env](#) [Install](#) [APIs](#) [Docker](#) [Run](#)

1 Prerequisites & Clone

Get the repository onto your machine



Clone the Repository

Download TradingAgents from GitHub

Start by cloning the official TradingAgents repository. Make sure you have **Git** installed on your system.

```
BASH

# Clone the repository
git clone https://github.com/TauricResearch/TradingAgents.git

# Navigate into the project
cd TradingAgents
```



Tip: If you want to contribute, fork the repo first and clone your fork instead.



Create Virtual Environment

Set up an isolated Python 3.12 environment

TradingAgents requires **Python 3.12**. Create a virtual environment using your preferred environment manager.



Conda **RECOMMENDED**



```
BASH

# Create environment
conda create --n tradingagents python=3.12 --y

# Activate it
conda activate tradingagents
```

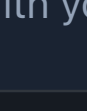


venv



```
BASH

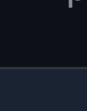
# Create & activate
python3.12 -m venv .venv
# macOS/Linux:
source .venv/bin/activate
# Windows:
# .venv\Scripts\activate
```



Python 3.12 installed



Virtual environment created



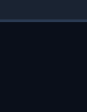
Environment is **activated**



Install the Package

pip install with all dependencies

With your virtual environment active, install TradingAgents and all its dependencies.



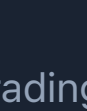
```
BASH

# Install the package and dependencies
pip install .

# Or in editable/development mode
# pip install --e ".[dev]"
```



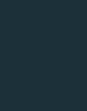
Need Bedrock support? Install with the bedrock extra: `pip install "[bedrock]"`



Package installed successfully (`pip install .`)



No errors in installation output



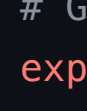
`tradingagents` command available



Set Your API Keys

Choose a provider and configure the key

TradingAgents supports multiple LLM providers. You need at least **one** API key to get started. If you have a Jio SIM, you can get a free Google Gemini Pro API key for **18 months**.



Free Gemini Pro — 18 Months (Jio SIM)

If you have an active **Jio SIM** in India, you can claim a **free Google AI Studio API key** (Gemini Pro) valid for **18 months**. Here's how:

- Open [Google AI Studio](#)
- Sign in with your Google account (the one linked to your Jio number)
- Click **Get API Key** → **Create API Key**
- Verify with your **Jio mobile number** when prompted
- Once verified, your free API key is generated — valid for **18 months**

Then set it as: `export GOOGLE_API_KEY="your-key-here"`



Method A — Environment Variables



Set at least ONE of the following:

```
# OpenAI (GPT)
export OPENAI_API_KEY="sk-..."

# Google (Gemini)
export GOOGLE_API_KEY="..."

# Anthropic (Claude)
export ANTHROPIC_API_KEY="sk-ant-..."

# xAI (Grok)
export XAI_API_KEY="..."

# DeepSeek
export DEEPSEEK_API_KEY="..."

# — International
export DASHSCOPE_API_KEY="..."

# — China
export DASHSCOPE_CN_API_KEY="..."

# GLM via Z.AI
export ZHIPU_API_KEY="..."

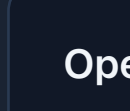
# GLM via (China)
export ZHIPU_CN_API_KEY="..."

# — Global
export MINIMAX_API_KEY="..."

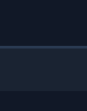
# — China
export MINIMAX_CN_API_KEY="..."

#
export OPENROUTER_API_KEY="..."

# Alpha Vantage (market data — optional)
export ALPHA_VANTAGE_API_KEY="..."
```

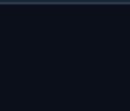


Method B — .env File



```
# Copy the example env file
cp .env.example .env

# Open .env in your editor and paste your keys
```

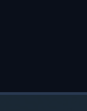


Other Provider Setups



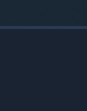
Azure OpenAI

Copy `.env.enterprise.example` to `.env.enterprise` and fill in credentials.



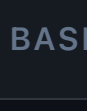
Ollama (Local)

Set `llm_provider`: "ollama". Default endpoint: `http://localhost:11434/v1`.



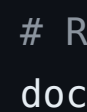
OpenAI-Compatible

Set `llm_provider`: "openai_compatible" with `backend_url` (vLLM, LM Studio, etc.).



AWS Bedrock

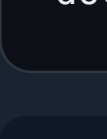
Install "[bedrock]", set AWS creds + region, use Bedrock model IDs.



At least one LLM provider API key configured



Alpha Vantage API key (set (market data) — optional)



Run with Docker

Isolated containerized environment

If you prefer containerized execution, Docker handles all dependencies automatically.

Standard Docker



```
BASH

# Copy and configure your API keys
cp .env.example .env

# Run the main container
docker compose run --rm tradingagents
```

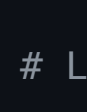


With Ollama (Local Models)

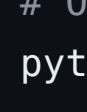


```
BASH

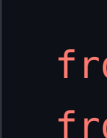
docker compose --profile ollama run --rm tradingagents-ollama
```



Docker & Docker Compose installed



`.env` file configured with API keys



Run the Framework

Start trading analysis from CLI or Python

TradingAgents offers two ways to run — through the interactive CLI or directly as a Python package.

Interactive CLI



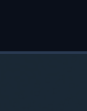
```
BASH

# Launch the interactive CLI
tradingagents

# Or run directly from source
python --m cli.main
```

The CLI will guide you through selecting tickers, analysis date, LLM provider, research depth, and more.

Python Package Usage



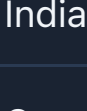
```
PYTHON

from tradingagents.graph.trading_graph import TradingAgentsGraph
from tradingagents.default_config import DEFAULT_CONFIG

# Initialize with defaults
ta = TradingAgentsGraph(debug=True, config=DEFAULT_CONFIG.copy())

# Run analysis for a ticker
_, decision = ta.propagate("NVDA", "2026-01-15")
print(decision)
```

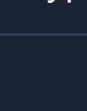
CLI with Checkpoint Resume



```
BASH

# Enable checkpoint resume (picks up where a crash left off)
tradingagents analyze --checkpoint

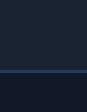
# Clear all checkpoints for a fresh run
tradingagents analyze --clear-checkpoints
```



CLI launches successfully (`tradingagents`)



Can select ticker, date, and LLM provider in the CLI



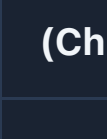
Python import works (`from tradingagents...`)



Supported Markets & Tickers

TradingAgents works with any market Yahoo Finance covers

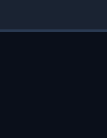
MARKET	EXAMPLE TICKERS
US	<code>AAPL</code> , <code>SPY</code>
Hong Kong	<code>0700.HK</code>
Tokyo	<code>7203.T</code>
London	<code>AZN.L</code>
India	<code>RELIANCE.NS</code> , <code>RELIANCE.BO</code>
Canada	<code>AAPL.TO</code>
Australia	<code>AAPL.AX</code>
China A-shares	<code>600519.SS</code> , <code>000001.SZ</code>
Crypto	<code>BTC-USD</code> , <code>ETH-USD</code>



LLM Provider Quick Reference

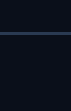
All supported providers and their API keys

PROVIDER	ENV VARIABLE	MODELS
OpenAI	<code>OPENAI_API_KEY</code>	GPT-5.x, GPT-4.x
Google	<code>GOOGLE_API_KEY</code>	Gemini 3.x
Anthropic	<code>ANTHROPIC_API_KEY</code>	Claude 4.x, Sonnet 5, Fable 5
xAI	<code>XAI_API_KEY</code>	Grok 4.x
DeepSeek	<code>DEEPSEEK_API_KEY</code>	DeepSeek V3
(Intl)	<code>DASHSCOPE_API_KEY</code>	2.5 / 3
(China)	<code>_CN_API_KEY</code>	via China
GLM (Z.AI)	<code>ZHIPU_API_KEY</code>	GLM International
GLM ()	<code>ZHIPU_CN_API_KEY</code>	GLM China
(Global)	<code>MINIMAX_API_KEY</code>	models
(China)	<code>MINIMAX_CN_API_KEY</code>	China
	<code>OPENROUTER_API_KEY</code>	Access to 100+ models
Azure OpenAI	<code>.env.enterprise</code>	Azure-hosted GPT
AWS Bedrock	AWS creds + region	Claude via Bedrock
Ollama	None (local)	Any local Ollama model
OpenAI-Compatible	<code>OPENAI_COMPATIBLE_API_KEY</code>	vLLM, LM Studio, llama.cpp



Troubleshooting

Common issues and how to resolve them



Python version mismatch: Make sure you're using Python 3.12 specifically. Other versions may have dependency conflicts.



Virtual env not activated: Always activate your environment before running pip or tradingagents. Verify with `which python`.



Windows encoding issues: Set `PYTHONUTF8=1` in your environment variables if you see UTF-8 errors.



Lower temperature for reproducibility: Set temperature: `0.0` in config for more deterministic outputs. Note: reasoning models largely ignore temperature.